Project 1

FreeCell Solver

1 Introduction

FreeCell[1] is a popular solitaire game that uses a standard 52-card deck. It became widely known after being included as a free desktop game in early versions of MS Windows operating system. In a typical session, the deck is dealt face up into 8 cascades, with 4 free cells and 4 foundation. The objective is to move all cards to the foundations by suit, in ascending order from Ace to King, using the free cells and cascades strategically.

Figure 1: A screenshot of the Microsoft FreeCell game.

This game is quite suitable for practicing search algorithms, such as BFS, DFS, UCS, and A*. First, it is easily accessible, students can quickly grasp the rules and relate them to the problem. Next, unlike many other solitaire games, all cards are face-up from the beginning, with no hidden or any random elements, allowing us to treat the searching as a fully observable planning problem. Finally, defining states, actions, and transitions is straightforward, and admissible heuristics can be constructed, making this an excellent problem for designing and analyzing heuristic search methods like A*.

[1] FreeCell — Wikipedia: https://en.wikipedia.org/wiki/FreeCell

2 Gameplay

The following Freecell rules are taken from Wikipedia. You can try the game via this link.

2.1 Construction and layout

One standard 52-card deck is used for the game. There are 4 empty cells and 4 empty foundations. Cards are dealt face-up into 8 cascades (columns), 4 columns of 7 cards and 4 columns of 6 cards.

2.2 Moves

Any cell card or top card of any cascade may be moved to build on a tableau, or moved to an empty cell, an empty cascade, or its foundation. Complete or partial tableaus may be moved to build on existing tableaus, or moved to empty cascades, by recursively placing and removing cards through intermediate locations.

- Move a single card from the top of a cascade to:
  - An empty free cell.
  - The top of another cascade if it forms a valid descending sequence of alternating colors (e.g., 7♠ on 8♡).
- Move a card from a cascade or a free cell to a foundation pile if:
  - It matches the suit of that foundation and
  - It is exactly one rank higher than the current top card of that foundation (starting from Ace up to King).
- Allow moving sequences of cards between cascades as long as:
  - The sequence itself is a valid descending alternating-color sequence, and
  - Its length does not exceed the limit imposed by the number of empty free cells and empty cascades (i.e., the sequence can be thought of as being moved one card at a time under standard FreeCell rules).

2.3 Victory

The game is won after all 52 cards have been moved to the foundations in correct order.

3 Requirements

Before we get started, here are some important things to consider:

- The source code must be written in **Python**.
- You may use supporting libraries (e.g., for GUI, data structures), but you must implement the search algorithms **yourself**.
- There is **no restriction** on how you organize your code. However, it should be professionally and clearly structured.

3.1 Reimplement the Game (5 pts)

Implement a basic, playable version of the FreeCell game. The program **MUST HAVE** a GUI; either a web application or a desktop application is acceptable.

F.Y.I:

- FreeCell games in Microsoft were generated by a fixed shuffling algorithm using a 15-bit seed, numbered from 1 to 32,000 (later expanded to millions of deals). Almost all of these games are solvable.[2]
- FreeCell Game Solutions is a website that provides structured solutions for a million standard deals based on Microsoft's FreeCell shuffling algorithm mentioned above. The site lists games that are rated as easy or difficult and provides solutions in multiple formats: Text form, Detailed step-by-step view, and Video demonstration.

The program must be able to display in the GUI the basic controls (e.g., New Game, Restart, and buttons for invoking the following solvers).

3.2 BFS Solver (10 pts)

- Implement a FreeCell solver using **Breadth-First Search (BFS)**.
- The solver must record the Search Time, Memory Usage, Expanded Nodes and Search Length for later analysis in the experiments section. The same result should be recorded for each of the following solvers.

[2] https://www.solitairelaboratory.com/fcfaq.html

3.3 DFS Solver (10 pts)

- Implement a FreeCell solver using **Depth-First Search (DFS)**.
- You may use **Iterative Deepening Search (IDS)** as an alternative if you prefer; clearly state this in your report.
- You should handle repeated states to avoid getting stuck in cycles (for example, using a visited set or hashing states).

3.4 UCS Solver (15 pts)

- Implement a FreeCell solver using **Uniform-Cost Search (UCS)**.
- *In the report*: Clearly **define and justify your cost function** in the report. For example:
  - Each move has cost 1 (minimize the number of moves), or
  - Different types of moves have different costs (e.g., moving to foundations is "cheap", moving into a free cell is "expensive"), as long as you provide a reasonable explanation.

3.5 A* Solver (15 pts)

- Implement a FreeCell solver using **A* Search** with an **appropriate heuristic** that estimates how close a given state is to the goal.
- Some non-mandatory heuristic ideas:
  - The number of cards not yet placed on the foundations.
  - Penalties for "important" low-rank cards (A, 2, 3, etc.) being buried deep inside the cascades.
  - Rewards or penalties based on how many free cells and empty cascades are available (more empty spaces usually mean the position is easier).
- *In the report*:
  - Explain how you designed your heuristic.
  - Discuss whether your heuristic is *admissible* (never overestimates true remaining cost) and/or *consistent*, with clear arguments.

3.6 Report (30 pts)

Submit a well-formatted PDF report including:

- **Project Planning and Task Distribution**:
  - Document the responsibilities of each team member, including which tasks were assigned and each member's completion percentage.
  - Example: if the group score is 9.0 and Student A has a completion percentage of 90%, then A's individual score is $9.0 \times 90\% = 8.1$.
  - It is important to evaluate contributions **fairly** and **accurately**.
- **Algorithm Description (10 pts)**:
  - Provide detailed explanations of each search algorithm used (BFS, DFS/IDS, UCS, A*).
  - Describe how you formalize the search problem: state representation, actions, goal test, cost function, heuristics, and how you avoid revisiting states.
- **Experiments (20 pts)**:
  - Evaluate search performance on multiple test cases (use a fixed test set to ensure that the experiments are fair across algorithms).
  - For each algorithm, measure:
    * Search time.
    * Memory usage (peak memory).
    * Number of expanded nodes.
    * Solution length (number of moves).
  - Present results with:
    * Tables summarizing the measurements.
    * Charts (plots) comparing the algorithms.
    * Your insights and discussion of the strengths and weaknesses of each algorithm.
- **References** and **Appendix**:
  - List all references (papers, books, online tutorials, GitHub repositories, etc.).
  - Include your Appendix, for example: how you used Generative AI (LLMs) in your work, what prompts you used.

3.7 Video (10 pts)

Upload a demo video to YouTube and provide the **public link** in your report. The video includes:

- **Graphical Interface (5 pts)**:
  - Demonstrate the GUI of your program.
  - Clearly show the detailed layout: cascades, free cells, foundations, and control buttons (e.g., New Game, Undo, Solve BFS/DFS/UCS/A*).
- **Feature Presentation (5 pts)**: Demonstrate the implemented functionalities:
  - Playing FreeCell manually (moving cards, winning/losing a game).
  - Running your solvers (e.g., choose a deal, run BFS/UCS/A*, show the found solution, possibly by animating or replaying the moves).

The video should contain subtitles or narration to make it easy to follow (either in English or Vietnamese).

4 Submission

Please follow to the following submission guidelines:

- Your source code and report must be contributed in the form of a compressed file (.zip, .rar) and named according to the format `StudentID1_StudentID2_...`
- If the compressed file is larger than 25MB, upload it to Google Drive and share it via a link. **Absolutely no modifications are allowed after the deadline**.

Example details of the directory organization:

```
StudentID1_StudentID2_...
├── Source
│   ├── main.py
│   ├── README.md (how to run source code)
│   ├── requirements.txt (libraries to be installed)
│   └── ...
└── Report.pdf (included demo video URLs)
```

5 Notices

Please pay attention to the following notices:

- This is a **GROUP** assignment.
- Duration: about 3 weeks.
- AI tools are **not restricted**; however, students should use them wisely. Lab instructors have the right to conduct additional oral interviews with random groups to assess their knowledge of the project.
- Any form of plagiarism, dishonesty, or misconduct will result in a grade of zero for the course.

The end.